

Guia de publicação do SYSDAE (Laravel 13)

Este guia leva o sistema do seu computador de desenvolvimento para um servidor **Ubuntu 24.04**, com **Apache, PHP 8.3 (FPM), MySQL 8 e HTTPS**. Ele não foi executado por mim: use-o em um servidor de teste primeiro e confira cada passo.

Troque os valores de exemplo (`sysdae.exemplo.com.br`, `SENHA_FORTE_AQUI`) pelos seus.

0. Antes de publicar (ainda no computador de desenvolvimento)

0.1 Acertar o fuso horário (importante)

Hoje o sistema usa o fuso UTC. Os horários gravados e a "data de hoje" ficam 3 horas à frente do horário de Brasília, e depois das 21h o dia sugerido em faltas e atendimentos já aparece como o dia seguinte. Corrija **antes** de gravar dados reais:

```
bash

grep -n "timezone" config/app.php
```

Se a linha for `'timezone' => 'UTC',`, troque por `'timezone' => 'America/Sao_Paulo',`. Se ela usar `env('APP_TIMEZONE', 'UTC')`, basta colocar `APP_TIMEZONE=America/Sao_Paulo` no `.env`. Depois rode `php artisan test` para confirmar que tudo continua passando.

0.2 Remover os arquivos de montagem

Os scripts usados para montar o sistema não devem ir para o servidor:

```
bash

# veja o que será removido
ls etapa*.php ajustar_*.php corrigir_*.php setup_sysdae.php 2>/dev/null
find . -path ./vendor -prune -o \( -name '*.bak' -o -name '*.novo.php' \) -print

# remova (confirmando um a um)
rm -i etapa*.php ajustar_*.php corrigir_*.php setup_sysdae.php
find . -path ./vendor -prune -o \( -name '*.bak' -o -name '*.novo.php' \) -print
```

0.3 O que NÃO deve ser copiado para o servidor

`.env`, `vendor/`, `node_modules/` e o conteúdo de `storage/` (fotos, anexos, logs) do seu computador. O servidor cria o seu próprio `.env` e instala o `vendor/` sozinho.

Mas copie o arquivo `public/.htaccess`: o Apache depende dele para direcionar as páginas ao Laravel (ele vem com o framework e é um arquivo "escondido", por isso é fácil de esquecer em cópias manuais).

Se usar Git, confira que o `.gitignore` tem `.env` e `/vendor`.

1. Preparar o servidor

Conecte por SSH e instale os pacotes:

```
bash
```

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y apache2 mysql-server unzip git composer \
  php8.3-fpm php8.3-cli php8.3-mysql php8.3-mbstring php8.3-xml \
  php8.3-curl php8.3-zip php8.3-bcmath php8.3-intl php8.3-gd
```

Instale o `php8.3-fpm` e **não** o `libapache2-mod-php8.3`. O guia usa o PHP-FPM, que é mais seguro e rápido com o Apache. Se o `libapache2-mod-php8.3` já estiver instalado, remova-o antes: `sudo apt remove libapache2-mod-php8.3`.

Firewall (libera só SSH e web; **não** abra a porta 3306 do MySQL):

```
bash
```

```
sudo ufw allow OpenSSH
sudo ufw allow 'Apache Full'
sudo ufw enable
```

Atualizações de segurança automáticas:

```
bash
```

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

2. Banco de dados MySQL

```
bash
```

```
sudo mysql_secure_installation
sudo mysql
```

Dentro do MySQL:

```
sql
```

```
CREATE DATABASE sysdae CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Usuário do sistema: só acessa o banco sysdae
CREATE USER 'sysdae_app'@'localhost' IDENTIFIED BY 'SENHA_FORTE_AQUI';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, DROP, REFERENCES
    ON sysdae.* TO 'sysdae_app'@'localhost';

-- Usuário só para backup
CREATE USER 'sysdae_backup'@'localhost' IDENTIFIED BY 'OUTRA_SENHA_FORTE';
GRANT SELECT, SHOW VIEW, TRIGGER, LOCK TABLES ON sysdae.* TO 'sysdae_backup'@'localhost';

FLUSH PRIVILEGES;
EXIT;
```

As permissões `(CREATE, ALTER, INDEX, DROP, REFERENCES)` do usuário do sistema servem para rodar as migrations nas atualizações. Se preferir um nível a mais de proteção, crie um terceiro usuário só para as migrations e deixe o `(sysdae_app)` apenas com `(SELECT, INSERT, UPDATE, DELETE)`.

3. Colocar o código no servidor

```
bash

sudo mkdir -p /var/www/sysdae
sudo chown $USER:www-data /var/www/sysdae
cd /var/www/sysdae
```

Copie o projeto (já sem os arquivos do item 0): por `(git clone)`, ou do seu computador:

```
bash

rsync -av --exclude='.env' --exclude='vendor' --exclude='node_modules' \
    --exclude='storage/logs/*' --exclude='storage/app/*' \
    ./ usuario@servidor:/var/www/sysdae/
```

Permissões (o Apache roda como o usuário `(www-data)`):

```
bash

cd /var/www/sysdae
sudo chown -R $USER:www-data .
sudo find . -type f -exec chmod 664 {} \;
sudo find . -type d -exec chmod 775 {} \;
sudo chmod -R ug+rwX storage bootstrap/cache
```

Confira que o `(.htaccess)` chegou:

```
bash
```

```
ls -la public/.htaccess
```

4. Configuração de produção (.env)

```
bash
```

```
cp .env.example .env
```

```
nano .env
```

Ajuste, no mínimo:

```
ini
```

```
APP_NAME=SYSDAE
```

```
APP_ENV=production
```

```
APP_DEBUG=false
```

```
APP_URL=https://sysdae.exemplo.com.br
```

```
APP_LOCALE=pt_BR
```

```
APP_TIMEZONE=America/Sao_Paulo
```

```
LOG_CHANNEL=daily
```

```
LOG_LEVEL=warning
```

```
LOG_DAILY_DAYS=14
```

```
DB_CONNECTION=mysql
```

```
DB_HOST=127.0.0.1
```

```
DB_PORT=3306
```

```
DB_DATABASE=sysdae
```

```
DB_USERNAME=sysdae_app
```

```
DB_PASSWORD=SENHA_FORTE_AQUI
```

```
SESSION_DRIVER=database
```

```
SESSION_SECURE_COOKIE=true
```

Proteja o arquivo:

```
bash
```

```
chmod 640 .env
```

Nunca deixe `APP_DEBUG=true` em produção: as telas de erro mostram trechos de código e dados internos.

5. Instalar e preparar o sistema

```
bash
```

```
cd /var/www/sysdae
composer install --no-dev --optimize-autoloader
php artisan key:generate --force
php artisan migrate --force
php artisan db:seed --class=PerfisSeeder --force
php artisan sysdae:permissoes-sincronizar
php artisan storage:link
php artisan optimize
```

Opcionais (rode só se o comando existir na sua versão: `php artisan list | grep filament`):

```
bash
```

```
php artisan filament:optimize
php artisan icons:cache
```

Primeiro administrador

O primeiro usuário ainda é criado pelo terminal. Depois, o resto é feito pela tela **Usuários**:

```
bash
```

```
php artisan make:filament-user
php artisan sysdae:perfil email-do-admin@exemplo.com admin
```

6. Limites de envio de arquivos (anexos e fotos)

Crie o arquivo `/etc/php/8.3/fpm/conf.d/99-sysdae.ini`:

```
ini
```

```
upload_max_filesize = 12M
post_max_size = 14M
memory_limit = 256M
```

```
bash
```

```
sudo systemctl restart php8.3-fpm
```

O Apache não limita o tamanho dos envios por padrão, então os limites acima (do PHP) são os que valem.

7. Apache

7.1 Ativar os módulos necessários

```
bash
```

```
sudo a2enmod rewrite headers proxy_fcgi setenvif
sudo a2enconf php8.3-fpm
```

- `rewrite`: o `.htaccess` do Laravel usa para direcionar todas as páginas ao `index.php`.
- `headers`: cabeçalhos de segurança.
- `proxy_fcgi` e a configuração `php8.3-fpm`: ligam o Apache ao PHP-FPM.

7.2 Criar o site

Crie `/etc/apache2/sites-available/sysdae.conf`:

```
apache
```

```
<VirtualHost *:80>
    ServerName sysdae.exemplo.com.br
    DocumentRoot /var/www/sysdae/public

    <Directory /var/www/sysdae/public>
        AllowOverride All
        Require all granted
        Options -Indexes
    </Directory>

    Header always set X-Frame-Options "SAMEORIGIN"
    Header always set X-Content-Type-Options "nosniff"

    ErrorLog ${APACHE_LOG_DIR}/sysdae-error.log
    CustomLog ${APACHE_LOG_DIR}/sysdae-access.log combined
</VirtualHost>
```

O `DocumentRoot` aponta para a pasta **public**. Nunca aponte para a raiz do projeto: isso expõe o `.env` e o código. O `AllowOverride All` é o que faz o `public/.htaccess` do Laravel funcionar.

7.3 Ativar o site e testar

```
bash
```

```
sudo a2dissite 000-default
sudo a2ensite sysdae
sudo apache2ctl configtest
sudo systemctl reload apache2
```

O `configtest` deve responder `Syntax OK`.

7.4 Se algo não funcionar

Sintoma	Causa provável	O que fazer
Só a página inicial abre; as demais dão 404	<code>rewrite</code> desativado ou <code>.htaccess</code> ausente	<code>sudo a2enmod rewrite</code> , confira <code>public/.htaccess</code> e o <code>AllowOverride All</code> , recarregue o Apache
403 Forbidden	permissão ou <code>Require</code>	confira o <code>Require all granted</code> e as permissões do item 3
500	erro do sistema	veja <code>storage/logs/</code> e <code>/var/log/apache2/sysdae-error.log</code>
O navegador baixa o arquivo PHP em vez de executá-lo	PHP-FPM não ligado	confira <code>sudo a2enconf php8.3-fpm</code> , <code>systemctl status php8.3-fpm</code> e recarregue o Apache

8. HTTPS (Let's Encrypt)

O domínio precisa já apontar (DNS) para o servidor.

```
bash
```

```
sudo apt install -y certbot python3-certbot-apache
sudo certbot --apache -d sysdae.exemplo.com.br
sudo certbot renew --dry-run
```

O certbot cria o site HTTPS, redireciona o HTTP para ele e configura a renovação automática. Depois, confirme que o `.env` tem `APP_URL` com `https://`.

9. Backup (banco e arquivos)

O que precisa de backup: o **banco**, a pasta `storage/app/private` (anexos sensíveis) e `storage/app/public` (fotos).

Credenciais do backup, sem senha na linha de comando. Crie `/root/.sysdae-mysql.cnf`:

```
ini
```

```
[client]
user=sysdae_backup
password=OUTRA_SENHA_FORTE
```

```
bash
```

```
sudo chmod 600 /root/.sysdae-mysql.cnf
```

Script `/usr/local/bin/sysdae-backup.sh`:

```
bash
```

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
DESTINO=/var/backups/sysdae
```

```
DATA=$(date +%F)
```

```
mkdir -p "$DESTINO"
```

```
chmod 700 "$DESTINO"
```

```
mysqldump --defaults-extra-file=/root/.sysdae-mysql.cnf \  
  --single-transaction --no-tablespaces --routines sysdae \  
  | gzip > "$DESTINO/banco-$DATA.sql.gz"
```

```
tar -czf "$DESTINO/arquivos-$DATA.tar.gz" \  
  -C /var/www/sysdae/storage/app private public
```

```
# guarda 14 dias
```

```
find "$DESTINO" -type f -mtime +14 -delete
```

```
bash
```

```
sudo chmod 700 /usr/local/bin/sysdae-backup.sh
```

```
sudo /usr/local/bin/sysdae-backup.sh          # teste agora
```

```
sudo crontab -e
```

```
# adicione a linha:
```

```
0 2 * * * /usr/local/bin/sysdae-backup.sh
```

Três cuidados:

1. Os backups contêm dados pessoais e sensíveis: mantenha a pasta com permissão restrita e, se copiar para fora do servidor, **criptografe** a cópia.
2. Guarde uma cópia **fora do servidor**. Um backup só no mesmo disco não protege contra perda do disco.
3. **Teste a restauração** pelo menos uma vez (em outro servidor ou banco), porque backup que nunca foi restaurado não é garantia.

10. Atualizações do sistema

Rotina para publicar uma nova versão:

```
bash
```



```
cd /var/www/sysdae
sudo /usr/local/bin/sysdae-backup.sh      # backup antes
php artisan down
git pull                                # ou rsync da nova versão
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan optimize
php artisan up
```

Antes de publicar, no ambiente de desenvolvimento, rode `php artisan test` e só publique se tudo passar.

Se algo der errado: `php artisan down`, restaure o backup do banco e dos arquivos, volte o código anterior e `php artisan up`.

11. Conferência depois de publicar

- ☐ O site abre em **https** e o cadeado aparece.
- ☐ `curl -I https://sysdae.exemplo.com.br/.env` responde **403 ou 404** (nunca 200).
- ☐ `sudo apache2ctl -M | grep -E "rewrite|headers|proxy_fcgi"` mostra os três módulos.
- ☐ O login funciona e uma senha errada é recusada.
- ☐ Na **Auditoria**, aparecem "Entrou" e "Falha de login".
- ☐ Um usuário de cada perfil entra e vê só o que deve ver.
- ☐ Uma ocorrência **sigilosa** não aparece para o DAE Central.
- ☐ Envio de um anexo de 5 MB funciona, e o download também.
- ☐ A foto de um aluno aparece, e um relatório em PDF abre com acentos corretos.
- ☐ O backup rodou e o arquivo existe em `/var/backups/sysdae`.
- ☐ Os horários na Auditoria batem com o horário de Brasília.

12. Segurança no dia a dia

- Senhas fortes para o servidor, o MySQL e os usuários; acesso SSH por **chave**.
- Inative (não apague) quem deixou a instituição, na tela **Usuários**.
- Rode `composer audit` de vez em quando e atualize as dependências com cuidado, testando antes.
- Os logs do sistema ficam em `storage/logs` (14 dias) e os do Apache em `/var/log/apache2/`. Olhe-os se algo estranho acontecer.
- Mantenha o sistema operacional atualizado.

13. Se for hospedagem compartilhada (cPanel etc.)

- A **raiz do site** precisa apontar para a pasta `public`. Se a hospedagem não permitir, o `.env` e o código ficam expostos: nesse caso, prefira um servidor próprio (VPS).
- A maioria dessas hospedagens usa Apache, então o `public/.htaccess` do Laravel já

funciona. Confirme que o arquivo foi enviado.

- Confirme que a hospedagem oferece **PHP 8.3 ou mais novo**, as extensões listadas no item 1 e acesso por terminal (SSH) para rodar os comandos do `artisan`.
 - Os limites de upload, o backup e o HTTPS costumam ser configurados no painel da hospedagem.
 - Não há como garantir o isolamento que um servidor próprio dá aos dados sensíveis: vale conversar com a instituição.
-

14. Proteção de dados (LGPD)

O sistema guarda dados de saúde, atendimentos psicossociais e ocorrências disciplinares de estudantes, que são dados pessoais sensíveis. Antes de usar em produção, a instituição deve definir:

- **por quanto tempo** guardar cada tipo de registro, inclusive a auditoria;
- quem é o **responsável** pelos dados e como atender pedidos dos titulares (acesso, correção, exclusão);
- a **base legal** para o tratamento e o aviso aos responsáveis dos estudantes.

O sistema oferece o controle de acesso por perfil, o sigilo de registros, os anexos privados e a auditoria. As decisões acima são da instituição e precisam de orientação jurídica própria.